PART 14–17 • ENTERPRISE INTEGRATION

Enterprise GitHub, APIs, Apps & CLI

Identity federation, REST/GraphQL APIs, GitHub Apps architecture, and CLI automation

---

TOPICS COVERED

- › GitHub Enterprise Cloud vs Server
- › RBAC & Repository Governance
- › Data Residency & Backup
- › REST API Architecture
- › Webhook Architecture & Events
- › GitHub Apps Architecture
- › OAuth Apps vs GitHub Apps
- › gh CLI Scripting & Automation
- › Release & PR Automation
- › SSO, SAML, SCIM, OIDC
- › Compliance & Audit Logs
- › Disaster Recovery & HA
- › GraphQL API & Analytics
- › Fine-Grained PATs
- › Installation Tokens & JWT
- › Deploy Keys & Machine Users
- › GitHub CLI Extensions
- › Enterprise Administration via CLI

PART 14 — Enterprise GitHub

14.1 GitHub Enterprise Cloud vs Server

Feature	GHEC	GHES
Hosting	GitHub-managed (Azure)	Customer-managed
Latest features	Immediate	1-3 month lag
SAML/OIDC SSO	Yes	Yes
SCIM provisioning	Yes	Yes (limited)
Data residency	US, EU, AU options	Full customer control
Network isolation	No (multi-tenant SaaS)	Complete (on-prem/VPC)
GitHub Connect	N/A (is cloud)	Connect GHES to GHEC
Actions runners	Hosted + self-hosted	Self-hosted only (or Actions proxy)
Backup	GitHub SLA	GitHub Enterprise Backup Utilities
Disaster recovery	GitHub handles	Customer handles
Migration path	Simpler (less infra)	Complex (upgrade cycles)
Cost model	Per-user/month	License + hosting
Support	GitHub Enterprise Support	GitHub Enterprise Support + Infra

14.2 Identity — SSO, SAML, SCIM, OIDC

SAML SSO

SAML (Security Assertion Markup Language) 2.0 is the standard for enterprise GitHub authentication. When enabled, users must authenticate via the organization's Identity Provider (Okta, Azure AD, Ping, OneLogin) to access organization resources.

```
# SAML Configuration checklist:
# In GitHub: Settings > Authentication security > SAML SSO
# Configure:
#   - Sign-on URL: https://mycompany.okta.com/app/github/sso/saml
#   - Issuer: https://mycompany.okta.com
#   - Public certificate: [X.509 cert from IdP]

# Important behaviors:
# - Users must link their GitHub account to SAML identity
# - Active sessions are checked on each API call (optional: can be periodic)
# - Machine users (bots) need their own SAML-authorized tokens
# - Fine-grained PATs and GitHub Apps support SAML SSO (need authorization)

# Authorize an existing PAT for SAML:
# GitHub.com > Settings > Developer settings > PATs >
#   Click PAT > "Configure SSO" > Authorize for org

# Check SAML-linked external identities:
```

```
gh api orgs/myorg/members --paginate \
  --jq '.[] | {login: .login, id: .id}'
```

SCIM Provisioning

SCIM (System for Cross-domain Identity Management) automates user provisioning and deprovisioning. When an employee is onboarded in Okta/Azure AD, they are automatically added to GitHub organizations. When offboarded, they are automatically removed.

```
# SCIM lifecycle:
# 1. IdP creates user → GitHub creates/links account, adds to org
# 2. IdP updates user → GitHub syncs attributes (name, email)
# 3. IdP deactivates user → GitHub removes from org (not from GitHub.com)
# 4. IdP deletes user → GitHub revokes org membership, removes SAML link

# SCIM-managed attributes:
# - userName (GitHub login)
# - displayName
# - emails
# - active (deactivation triggers org removal)
# - groups → GitHub Teams (group push)

# Team sync with IdP groups:
# Settings > Teams > [Team] > Settings > Team sync
# Map IdP group "platform-engineers" to GitHub team "platform-engineers"
# Members added/removed in IdP are synced to GitHub team automatically
```

Enterprise Managed Users (EMUs)

EMUs are a special GHEC configuration where ALL user accounts are managed by the enterprise. Users cannot create personal GitHub accounts — their accounts are owned by the enterprise and prefixed with the enterprise slug (user@enterprise).

- Strongest enterprise control — no personal account mixing
- Users cannot have public repositories or contribute to public repos
- Requires full SCIM provisioning — no manual account creation
- Best for regulated industries requiring strict identity governance

> ■ *EMUs are a significant commitment — migrating back to standard accounts is complex. Evaluate carefully before adopting, especially if your engineers contribute to open source.*

14.3 Audit Logs

GitHub Enterprise provides comprehensive audit logs for all events: repository creation/deletion, permission changes, workflow runs, secret access, SSO events, and more.

```
# Access audit log via API:
gh api /enterprises/myenterprise/audit-log \
  --paginate \
  --jq '.[] | select(.action | startswith("repos"))' \
  | jq '.'

# Stream audit log to SIEM (Splunk, Datadog, etc.):
# GitHub supports streaming to:
# - AWS S3 / CloudWatch
# - Azure Blob / Event Hubs
# - Google Cloud Storage
# - Splunk
# - Datadog

# Configure streaming (via API):
gh api /enterprises/myenterprise/audit-log/streaming \
  --method PUT \
```

```
  --field enabled=true \
  --field vendor_name=splunk \
  --field url="https://inputs.example.splunkcloud.com:8088" \
  --field token="splunk-hec-token"

# Key audit events to monitor:
# org.add_member / org.remove_member
# repo.create / repo.destroy / repo.transfer
# protected_branch.policy_override  ← admin bypassed branch protection
# secret.access                      ← secret accessed
# workflow.run                       ← workflow execution
# two_factor_authentication.disabled ← security regression
```

14.4 Backup and Disaster Recovery

```
# GitHub Enterprise Backup Utilities (GHES only):
# https://github.com/github/backup-utils

# Install:
git clone https://github.com/github/backup-utils
cp backup.config-example backup.config

# Configure backup.config:
GHE_HOSTNAME="github.mycompany.com"
GHE_BACKUP_BUCKET="/mnt/backups"
GHE_NUM_SNAPSHOTS=10    # Keep 10 snapshots
GHE_RSYNC_OPTS=""

# Run backup (captures: repositories, settings, MySQL, Elasticsearch, audit logs):
bin/ghe-backup

# Restore to standby/replacement appliance:
bin/ghe-restore github.mycompany-dr.com

# High availability setup:
# Primary appliance → replicates to replica via ghe-repl
# Automatic failover: ~30-60 seconds with DNS update
# Set up replica:
ghe-repl-setup github.mycompany-replica.com
ghe-repl-start

# Check replication status:
ghe-repl-status
```

PART 15 — GitHub APIs

15.1 REST API

GitHub's REST API follows REST conventions with resource-based URLs. The current version is v3 (implied in the Accept header).

```
# Base URL: https://api.github.com
# GHES: https://github.mycompany.com/api/v3

# Authentication:
curl -H "Authorization: Bearer $GITHUB_TOKEN" \
     -H "Accept: application/vnd.github+json" \
     -H "X-GitHub-Api-Version: 2022-11-28" \
     https://api.github.com/repos/myorg/myrepo

# Pagination (link header):
# Link: <https://api.github.com/repos/...?page=2>; rel="next",
#       <https://api.github.com/repos/...?page=5>; rel="last"

# With gh CLI:
gh api /repos/myorg/myrepo/issues --paginate --jq '.[].title'

# Rate limits:
# Authenticated REST: 5,000 req/hour (per token)
# GitHub App: 5,000-15,000/hour depending on installation
# Search API: 30 req/min (authenticated)
# Check your rate limit:
gh api /rate_limit
```

Key REST API Endpoints

Area	Key Endpoints
Repos	GET /repos/{owner}/{repo}, POST /orgs/{org}/repos
Commits	GET /repos/{owner}/{repo}/commits, GET /commits/{sha}
Branches	GET /repos/{owner}/{repo}/branches, DELETE /refs/heads/{branch}
Pull Requests	POST /pulls, GET /pulls/{number}, PATCH /pulls/{number}/merge
Actions	GET /actions/runs, POST /actions/workflows/{id}/dispatches
Releases	POST /releases, GET /releases/latest, PATCH /releases/{id}
Deployments	POST /deployments, POST /deployments/{id}/statuses
Webhooks	POST /repos/{owner}/{repo}/hooks

15.2 GraphQL API

GitHub's GraphQL API (api.github.com/graphql) enables fetching exactly the data needed in a single request — critical for analytics dashboards and tooling that would otherwise require dozens of REST calls.

```
# GraphQL query example — repository metrics:
query OrgDashboard($org: String!, $cursor: String) {
  organization(login: $org) {
    repositories(first: 100, after: $cursor, orderBy: {field: UPDATED_AT, direction: DESC}) {
```

```
      pageInfo {
        hasNextPage
        endCursor
      }
      nodes {
        name
        defaultBranchRef {
          name
          target {
            ... on Commit {
              committedDate
              author { name email }
              checkSuites(first: 1) {
                nodes {
                  status
                  conclusion
                  workflowRun {
                    workflow { name }
                    runNumber
                  }
                }
              }
            }
          }
        }
        vulnerabilityAlerts(first: 1) {
          totalCount
        }
        secretScanningAlerts(first: 1) {
          totalCount
        }
      }
    }
  }
}

# Run with gh:
gh api graphql -f query="$(cat query.graphql)" \
  -F org=myorg | jq '.data.organization.repositories.nodes[] |
    {name: .name, alerts: .vulnerabilityAlerts.totalCount}'
```

Webhook Architecture

Webhooks deliver real-time event notifications to your infrastructure via HTTP POST. GitHub sends events when actions happen (pushes, PRs, deployments, etc.).

```
# Webhook delivery guarantees:
# - At-least-once delivery (may retry on timeout/5xx)
# - Retry schedule: 0s, 1s, 5s, 30s, 120s, 3min, 10min, 30min, 1hr, 2hr, 4hr, 8hr
# - Secret header: X-Hub-Signature-256 (HMAC-SHA256)
# - Delivery timeout: 10 seconds (must respond quickly, then process async)

# Verify webhook signature (Python):
import hashlib, hmac

def verify_webhook(payload: bytes, signature: str, secret: str) -> bool:
    expected = "sha256=" + hmac.new(
        secret.encode(), payload, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, signature)

# Lambda handler pattern (respond fast, process async):
def handler(event, context):
    # 1. Verify signature immediately
    # 2. Push to SQS/SNS
    # 3. Return 200 within 10 seconds
    sqs.send_message(QueueUrl=QUEUE_URL, MessageBody=event['body'])
    return {"statusCode": 200}
```

PART 16 — GitHub Apps

16.1 GitHub Apps Architecture

GitHub Apps are the preferred way to integrate with GitHub. Unlike PATs, GitHub Apps are installations per-repository or per-organization, with fine-grained permissions, and do not consume a user's rate limit.

```
# GitHub App authentication flow:
# 1. Generate JWT signed with the App's private key:

import jwt, time

def get_app_jwt(app_id: str, private_key: str) -> str:
    now = int(time.time())
    payload = {
        "iat": now - 60,       # Issued at (60s clock skew)
        "exp": now + 600,      # Expires in 10 minutes (max 10min)
        "iss": app_id          # App ID
    }
    return jwt.encode(payload, private_key, algorithm="RS256")

# 2. Get installation access token:
# POST /app/installations/{installation_id}/access_tokens
# Headers: Authorization: Bearer <JWT>
# Returns: { "token": "ghs_xxxx", "expires_at": "..." }

# 3. Use installation token for API calls:
curl -H "Authorization: token ghs_xxxx" \
     https://api.github.com/repos/myorg/myrepo/pulls
```

Credential Comparison Matrix

Type	Rate limit	Scope	Expiry	Recommended for
Classic PAT	5k/hr per user	Coarse (repo, org)	Never (default)	Never (legacy)
Fine-grained PAT	5k/hr per user	Per-repo, fine-grained	Custom (max 1yr)	Developer personal use
OAuth App token	5k/hr per user	User-delegated	Never (until revoked)	User-facing integrations
GitHub App token	5k-15k/hr per install	Fine-grained, per-install	1 hour	Automation, bots, CI
GITHUB_TOKEN	1k/hr per repo	Workflow scope	Job lifetime	In-workflow only
Deploy Key	N/A (git only)	Single repo, read or write	Never	Read-only CI clone

■ *Never use classic PATs for automation. They are tied to a user account, have no expiry by default, have coarse scope, and if the user leaves the org, all automation breaks. Use GitHub Apps for all automation.*

GitHub App Permissions (Fine-Grained)

```
# Example App permissions configuration (app manifest):
{
  "name": "Platform Automation Bot",
  "description": "Manages CI/CD and release workflows",
  "permissions": {
    "contents": "write",          # Push commits, create releases
    "pull_requests": "write",     # Comment, review, approve
    "checks": "write",            # Create check runs
    "issues": "write",            # Comment, close issues
    "deployments": "write",       # Create deployment records
    "metadata": "read",           # Required for all Apps
    "packages": "write",          # Push to GHCR
```

```
    "statuses": "write"           # Update commit statuses
  },
  "events": [
    "pull_request",
    "push",
    "release",
    "check_suite"
  ],
  "webhook_url": "https://automation.mycompany.com/github/webhook",
  "public": false
}

# Using a GitHub App in Actions (generate token at runtime):
- uses: actions/create-github-app-token@v1
  id: app-token
  with:
    app-id: ${{ vars.APP_ID }}
    private-key: ${{ secrets.APP_PRIVATE_KEY }}
    owner: myorg

- uses: actions/checkout@v4
  with:
    token: ${{ steps.app-token.outputs.token }}
    # Now push commits will be authored by the App, not the user
```

PART 17 — GitHub CLI

17.1 gh CLI Overview

The GitHub CLI (gh) is the official command-line tool for GitHub. It enables scripting almost any GitHub operation without leaving the terminal, and integrates tightly with git for a seamless workflow.

```
# Install:
brew install gh              # macOS
winget install GitHub.cli    # Windows
sudo apt install gh          # Debian/Ubuntu

# Authenticate:
gh auth login                # Interactive
gh auth login --with-token < token.txt  # CI
GITHUB_TOKEN=xxx gh ...      # Env var

# Switch between hosts (GitHub.com + GHES):
gh auth login --hostname github.mycompany.com
gh config set git_protocol ssh

# Check auth status:
gh auth status
```

PR Automation

```
# Create PR with all options:
gh pr create \
  --title "feat: add OIDC authentication" \
  --body-file .github/pull_request_template.md \
  --base main \
  --head feature/oidc-auth \
  --reviewer alice,bob,@platform-team \
  --assignee @me \
  --label "enhancement,security" \
  --project "Platform Q3 Roadmap" \
  --milestone "v2.0.0"

# Merge PR with specific strategy:
gh pr merge 123 \
  --squash \
  --auto \
  --delete-branch \
  --body "Squashed: feat: add OIDC authentication"

# Batch operations across repos:
for repo in $(gh repo list myorg --limit 100 --json name -q '.[].name'); do
  gh pr list --repo "myorg/$repo" --state open \
    --json title,number,author --jq '.[] | {repo: "'"$repo"'", title, number}'
done
```

Release Automation

```
# Create a release with auto-generated notes:
gh release create v2.1.0 \
  --title "Release v2.1.0" \
  --generate-notes \
  --notes-start-tag v2.0.0 \
  ./dist/app-linux-amd64 \
  ./dist/app-darwin-arm64 \
  ./dist/sbom.spdx.json

# List releases and download assets:
gh release list --limit 10 | head -5
gh release download v2.0.0 --pattern '*.tar.gz' --dir ./downloads
# Edit a release (add assets post-publish):
```

```
gh release upload v2.1.0 ./dist/app-windows-amd64.exe
```

Enterprise Administration via CLI

```
# List all repositories in an org with metadata:
gh repo list myorg \
  --limit 1000 \
  --json name,isPrivate,isArchived,defaultBranchRef,updatedAt \
  --jq '.[] | select(.isArchived == false)' \
  | jq -r '[.name, .defaultBranchRef.name, .updatedAt] | @csv'

# Audit open PRs older than 30 days:
gh pr list --repo myorg/api-service \
  --state open \
  --json number,title,createdAt,author \
  --jq '.[] | select(.createdAt < (now - 2592000 | todate))'

# Apply a label to all repos in org:
gh repo list myorg --json name -q '.[].name' | while read repo; do
  gh label create "needs-review" \
    --color "FF6B35" \
    --description "Needs security review" \
    --repo "myorg/$repo" 2>/dev/null || true
done

# Run a workflow in all repos:
gh repo list myorg --json name -q '.[].name' | while read repo; do
  gh workflow run security-scan.yml \
    --repo "myorg/$repo" \
    --ref main 2>/dev/null || true
done

# Check which repos are missing a CODEOWNERS file:
gh repo list myorg --json name -q '.[].name' | while read repo; do
  exists=$(gh api "repos/myorg/$repo/contents/.github/CODEOWNERS" \
    --silent -q '.name' 2>/dev/null)
  [ -z "$exists" ] && echo "Missing CODEOWNERS: $repo"
done
```

gh CLI Extensions

gh CLI supports custom extensions written in any language, distributed as GitHub repositories prefixed with gh-.

```
# Install popular extensions:
gh extension install nicholasgasior/gh-report   # Org activity reports
gh extension install dlvhdr/gh-dash             # Dashboard TUI
gh extension install mislav/gh-branch           # Branch management
gh extension install yuler/gh-todo              # TODO list from issues

# Create your own extension:
gh extension create my-extension
cd gh-my-extension
# The entry point is any executable named gh-my-extension

# Example shell extension:
#!/bin/bash
# gh-security-audit: audit all repos for security settings
for repo in $(gh repo list $1 --json name -q '.[].name'); do
  branch_protection=$(gh api "repos/$1/$repo/branches/main/protection" 2>/dev/null)
  echo "$repo: $(echo $branch_protection | jq -r '.required_pull_request_reviews // "NONE"')"
done

chmod +x gh-my-extension
gh extension install .
```

Interview Questions — Enterprise, APIs & Apps

Q: What is the difference between a GitHub App and an OAuth App?

A: GitHub Apps are installations (scoped to repos/orgs) with fine-grained permissions, dedicated rate limits, short-lived tokens (1 hour), and are tied to the installation not a user. OAuth Apps act on behalf of a user, inherit the user's permissions, share the user's rate limit, and tokens don't expire. GitHub Apps are always preferred for automation because they don't depend on a user account and have better security scoping.

Q: Explain SCIM provisioning and why enterprises need it.

A: SCIM automates user lifecycle management between an Identity Provider (Okta, Azure AD) and GitHub. When HR creates an employee in the IdP, SCIM automatically adds them to GitHub and the correct teams. When they leave, SCIM removes their access immediately. Without SCIM, offboarding requires manual GitHub deprovisioning — a security risk in large organizations where manual processes are error-prone.

Q: What are the rate limits for the GitHub APIs, and how do GitHub Apps differ?

A: REST API authenticated requests: 5,000/hour per token. GraphQL API: 5,000 points/hour. GitHub Apps get installation-based limits: 5,000/hour baseline plus 50 requests per user per hour up to 15,000/hour for high-traffic installations. GITHUB_TOKEN in Actions: 1,000/hour per repo. Search API: 30/minute authenticated.

Q: How would you audit all repositories in an organization for missing branch protection?

A: Use the GitHub GraphQL API or REST API via 'gh api' with pagination: list all repos, then check /repos/{owner}/{repo}/branches/main/protection for each. A bash script using gh repo list + gh api can do this efficiently. For org-wide enforcement, use Repository Rulesets which apply to ALL repos matching a pattern without requiring per-repo configuration.